

Homework #4

Name(s): Raj Shah (ras637),

() ,

()

Problem 1

Suppose we have n persons with their heights given in an array $P[1 : n]$ and n skis with heights given in an array $S[1 : n]$. Design and analyze a greedy algorithm that in $O(n \log n)$ time assigns a ski to each person so that

$$\frac{1}{n} \sum_{i=1}^n |P[i] - S[\sigma(i)]|$$

is minimized.

Algorithm

Greedy idea: sort both lists and match the k th shortest person with the k th shortest ski.

Algorithm.

1. Create an array A of pairs $(P[i], i)$ for all people, where i is the original index.
2. Create an array B of pairs $(S[j], j)$ for all skis, where j is the original index.
3. Sort A by height and sort B by height in nondecreasing order.
4. For each $k = 1, 2, \dots, n$, assign the ski in $B[k]$ to the person in $A[k]$.
5. Build the permutation σ by setting

$$\sigma(A[k].\text{index}) = B[k].\text{index} \quad \text{for each } k.$$

6. Output σ .

Why this greedy strategy is correct.

Let the sorted people heights be

$$p_1 \leq p_2 \leq \dots \leq p_n$$

and the sorted ski heights be

$$s_1 \leq s_2 \leq \dots \leq s_n.$$

Our algorithm matches p_k with s_k for every k .

We prove that this minimizes the total cost

$$\sum_{k=1}^n |p_k - s_{\pi(k)}|$$

over all permutations π .

Lemma. For any $a \leq b$ and $x \leq y$,

$$|a - y| + |b - x| \geq |a - x| + |b - y|.$$

Proof. This is the uncrossing property for absolute values. Intuitively, if the smaller person is matched to the larger ski and the larger person is matched to the smaller ski, then the two assignments cross. Uncrossing them cannot increase the total absolute difference. Therefore the noncrossing pairing is always at least as good. $\square$

Now consider any optimal assignment. If it is not the sorted matching, then there exist indices $i < j$ such that p_i is matched to s_j and p_j is matched to s_i . By the lemma,

$$|p_i - s_j| + |p_j - s_i| \geq |p_i - s_i| + |p_j - s_j|.$$

So we can swap these two assignments and not increase the cost. Repeating this process removes all crossings, and eventually we obtain exactly the sorted matching. Hence the sorted matching is optimal.

Since minimizing the sum also minimizes the average (the factor $\frac{1}{n}$ is constant), the algorithm is correct.

Running time. Creating the two arrays takes $O(n)$. Sorting both arrays takes $O(n \log n)$. Reconstructing σ takes $O(n)$. Therefore the total running time is

$$O(n \log n).$$

Final Answer

Sort the people and skis by height and match them in the same sorted order. This produces an optimal assignment by an exchange argument: any crossing pair of matches can be uncrossed without increasing the total absolute difference. The running time is $O(n \log n)$.

Problem 2

Given a set of jobs, each with a deadline and a processing time, schedule the jobs to minimize the maximum lateness, where lateness of job j is completion time minus deadline.

Algorithm

Greedy rule: process jobs in nondecreasing order of deadlines.

Algorithm.

1. For each job j , let p_j be its processing time and d_j be its deadline.
2. Sort the jobs so that

$$d_1 \leq d_2 \leq \dots \leq d_n.$$

3. Execute the jobs in that order.
4. If C_j is the completion time of job j , define its lateness by

$$L_j = C_j - d_j.$$

5. Output the schedule and the value

$$L_{\max} = \max_j L_j.$$

Proof of correctness.

This is the *Earliest Deadline First* schedule. We prove it is optimal by an exchange argument.

Consider any schedule that is not sorted by deadlines. Then there exists an *inversion*: two consecutive jobs i and j such that job i is scheduled before job j , but

$$d_i > d_j.$$

Suppose these two jobs start at time t .

In the original order (i, j) , their completion times are

$$C_i = t + p_i, \quad C_j = t + p_i + p_j.$$

In the swapped order (j, i) , their completion times become

$$C'_j = t + p_j, \quad C'_i = t + p_j + p_i.$$

We compare maximum lateness of these two jobs before and after the swap.

For job j ,

$$L'_j = C'_j - d_j = t + p_j - d_j \leq t + p_i + p_j - d_j = L_j,$$

because $p_i \geq 0$.

For job i ,

$$L'_i = C'_i - d_i = t + p_i + p_j - d_i.$$

Since $d_i > d_j$, we have

$$L'_i = t + p_i + p_j - d_i \leq t + p_i + p_j - d_j = L_j.$$

So after swapping, both latenesses are at most L_j , and therefore

$$\max(L'_i, L'_j) \leq \max(L_i, L_j).$$

Thus swapping an inverted adjacent pair does not increase the maximum lateness.

By repeatedly removing inversions, any schedule can be transformed into deadline order without increasing maximum lateness. Therefore the deadline-ordered schedule is optimal.

Running time. Sorting takes $O(n \log n)$. Computing completion times and lateness values takes $O(n)$. So the total running time is

$$O(n \log n).$$

Final Answer

Schedule the jobs in nondecreasing order of deadlines. This minimizes the maximum lateness by an exchange argument on adjacent inverted jobs. The running time is $O(n \log n)$.

Problem 3

Alice wants to invite as many people as possible. For every invited guest, at least five other invited guests must be people they know, and at least five other invited guests must be people they do *not* know. Describe and analyze an algorithm to compute the largest possible guest list.

Graph model. Represent the n people by an undirected graph $G = (V, E)$:

- each vertex is a person,
- an edge (u, v) means u and v know each other.

For a set $S \subseteq V$, a person $v \in S$ is valid if:

$$\deg_S(v) \geq 5 \quad \text{and} \quad (|S| - 1) - \deg_S(v) \geq 5.$$

The second condition is equivalent to

$$\deg_S(v) \leq |S| - 6.$$

So we want the largest subset S such that every vertex in S satisfies

$$5 \leq \deg_S(v) \leq |S| - 6.$$

Algorithm

Algorithm.

1. Start with $S = V$.
2. For each vertex v , maintain its current degree inside S , denoted $\deg_S(v)$.
3. Put into a queue every vertex v that currently violates at least one condition:

$$\deg_S(v) < 5 \quad \text{or} \quad \deg_S(v) > |S| - 6.$$

4. While the queue is not empty:
 - (a) Remove a vertex v from the queue.
 - (b) If v is already deleted, ignore it.
 - (c) Otherwise delete v from S .
 - (d) For each remaining vertex u :
 - if $(u, v) \in E$, then decrement $\deg_S(u)$ by 1,
 - if $(u, v) \notin E$, then u has lost one non-neighbor in S , so the upper-bound condition may change because $|S|$ decreases.
 - (e) After the deletion, recheck affected vertices and insert into the queue any vertex that now violates

$$\deg_S(u) \geq 5 \quad \text{and} \quad \deg_S(u) \leq |S| - 6.$$

5. When the process stops, output the remaining set S .

Proof of correctness.

We show two facts:

Fact 1: At the end, the returned set S is valid.

This is immediate: the algorithm stops only when no remaining vertex violates either condition. Therefore every remaining guest knows at least 5 others in S and does not know at least 5 others in S .

Fact 2: Every deleted vertex can never belong to any feasible solution contained in the current set.

Suppose the algorithm deletes a vertex v while the current set is S .

- If $\deg_S(v) < 5$, then in any subset $T \subseteq S$ that still contains v , we have

$$\deg_T(v) \leq \deg_S(v) < 5.$$

So v cannot satisfy the first requirement inside any such subset.

- If $\deg_S(v) > |S| - 6$, then the number of people in S that v does *not* know is

$$(|S| - 1) - \deg_S(v) < 5.$$

In any subset $T \subseteq S$ containing v , the number of non-neighbors of v inside T can only stay the same or decrease, never increase. So v still has fewer than 5 non-neighbors in T , and therefore cannot satisfy the second requirement.

Thus every deleted vertex is impossible to keep in any feasible guest list contained in the current set.

Now let S^* be any feasible guest list. Since no deleted vertex can belong to a feasible subset of the current set, the algorithm never deletes a vertex from S^* . Therefore S^* is always contained in the current remaining set. When the algorithm terminates with final set S , we must have

$$S^* \subseteq S.$$

But S itself is feasible by Fact 1. Hence S is a feasible set containing every other feasible set, so it is the *largest possible* guest list.

Running time. A simple implementation using an adjacency matrix allows us to update and recheck vertices in $O(n)$ time per deletion, and at most n deletions occur. Therefore the total running time is

$$O(n^2).$$

If the graph is dense, this is fine. The problem only asked for an algorithm and analysis, and this bound is sufficient.

Final Answer

Repeatedly delete any person who currently knows fewer than 5 invited people or knows more than $|S| - 6$ invited people. This process ends with exactly the largest feasible guest set. A simple implementation runs in $O(n^2)$ time.

Problem 4

Given a directed graph $G = (V, E)$ and two vertices s and t , design an $O(n + m)$ algorithm that outputs Yes if there is a walk from s to t of even length, and No otherwise.

Algorithm

Idea: keep track of both the vertex and the parity of the walk length used to reach it.

Construct an implicit state graph with states

$$(v, 0), (v, 1) \quad \text{for each } v \in V,$$

where:

- $(v, 0)$ means we reached v by an even-length walk,
- $(v, 1)$ means we reached v by an odd-length walk.

For every directed edge $(u, v) \in E$, add transitions

$$(u, 0) \rightarrow (v, 1) \quad \text{and} \quad (u, 1) \rightarrow (v, 0).$$

Then run BFS or DFS from the start state $(s, 0)$. Output **Yes** iff the state $(t, 0)$ is reachable.

Proof of correctness.

We prove the following claim:

Claim. For every vertex v :

- state $(v, 0)$ is reachable in the state graph iff there exists an even-length walk from s to v in G ,
- state $(v, 1)$ is reachable in the state graph iff there exists an odd-length walk from s to v in G .

Proof. We use induction on walk length.

- Initially, $(s, 0)$ is reachable, corresponding to the length-0 walk from s to itself.
- Every time we traverse one edge of G , the parity flips. That is exactly what the transition rule in the state graph encodes.

So reachability in the state graph exactly matches existence of a walk with the corresponding parity in the original graph. $\square$

Therefore, there exists an even-length walk from s to t iff $(t, 0)$ is reachable from $(s, 0)$. Hence the algorithm is correct.

Running time. The state graph has $2n$ states and $2m$ transitions. BFS or DFS on this graph takes

$$O(n + m).$$

Final Answer

Run BFS/DFS on the parity-expanded graph with states (v, even) and (v, odd) . Start from (s, even) and answer Yes exactly when (t, even) is reachable. The running time is $O(n + m)$.

Problem 5

We have an undirected graph $G = (V, E)$, two vertices s, t , and a list of new edges F that are not already in E . For each $f \in F$, let G_f be the graph obtained by adding f to G . Design an algorithm in $O(n + m + |F|)$ time that finds the edge $f \in F$ minimizing the distance from s to t in G_f .

Algorithm

Algorithm.

1. Run BFS from s in G and compute

$$d_s(v) = \text{dist}_G(s, v) \quad \text{for all } v \in V.$$

2. Run BFS from t in G and compute

$$d_t(v) = \text{dist}_G(v, t) \quad \text{for all } v \in V.$$

3. For each candidate edge $f = (u, v) \in F$, compute

$$D_f = \min\{d_s(u) + 1 + d_t(v), d_s(v) + 1 + d_t(u)\}.$$

4. Return an edge $f \in F$ with minimum value of D_f .

Why this works.

Fix a candidate edge $f = (u, v) \in F$. Any shortest s - t path in G_f either:

- does not use the new edge f , in which case it is already a path in G , or
- uses f exactly once.

The second statement is enough because if a shortest path used f more than once, then it would repeat vertices or edges unnecessarily and could be shortened.

If the path uses f , then since the graph is undirected, the path must have one of these two forms:

$$s \rightsquigarrow u \rightarrow v \rightsquigarrow t \quad \text{or} \quad s \rightsquigarrow v \rightarrow u \rightsquigarrow t.$$

The shortest such path therefore has length

$$\min\{d_s(u) + 1 + d_t(v), d_s(v) + 1 + d_t(u)\}.$$

So D_f is exactly the best possible distance from s to t among paths that use the newly added edge f .

Now compare all edges in F . The edge with minimum D_f is exactly the edge whose addition gives the smallest possible s - t distance among all graphs G_f .

Running time. Each BFS takes $O(n + m)$. Evaluating all edges in F takes $O(|F|)$. Hence the total running time is

$$O(n + m + |F|).$$

Final Answer

Run BFS once from s and once from t . For each candidate edge $(u, v) \in F$, compute

$$\min\{d_s(u) + 1 + d_t(v), d_s(v) + 1 + d_t(u)\}.$$

Choose the edge with the smallest value. The total running time is $O(n + m + |F|)$.

Problem 6

In a directed graph, every edge is colored red, white, or blue. A walk is a *French flag walk* if its edge colors are

red, white, blue, red, white, blue, ...

Describe an algorithm to find all vertices reachable from a given start vertex v by a French flag walk.

Algorithm

Idea: the next allowed color depends only on the current step number modulo 3.

Create an implicit state graph with states

$$(u, 0), (u, 1), (u, 2) \quad \text{for each } u \in V,$$

where:

- state $(u, 0)$ means the next edge must be red,
- state $(u, 1)$ means the next edge must be white,
- state $(u, 2)$ means the next edge must be blue.

Start from $(v, 0)$ and do BFS/DFS:

- from $(u, 0)$, follow only red outgoing edges (u, w) and go to $(w, 1)$,
- from $(u, 1)$, follow only white outgoing edges (u, w) and go to $(w, 2)$,
- from $(u, 2)$, follow only blue outgoing edges (u, w) and go to $(w, 0)$.

At the end, output every vertex u such that at least one of $(u, 0)$, $(u, 1)$, or $(u, 2)$ is reachable.

Proof of correctness.

A French flag walk is completely determined by two things:

- the current vertex,
- which color must be used next.

That is exactly what the state (u, r) stores.

We prove the invariant:

Claim. A state (u, r) is reachable in the state graph iff there exists a French flag walk from the start vertex v to u such that the next required color corresponds to residue r .

Proof. Initially, $(v, 0)$ is reachable, corresponding to the empty walk at v , where the first edge must be red. Every valid move in the state graph follows exactly one allowed color and updates the required next color cyclically. So state-graph reachability is exactly the same as reachability by a French flag walk in the original graph. $\square$

Therefore a vertex u is output exactly when there exists some French flag walk from v to u .

Running time. There are $3n$ states and at most $3m$ transitions. So the traversal runs in

$$O(n + m).$$

Final Answer

Use BFS/DFS on a graph of states $(u, 0), (u, 1), (u, 2)$ that remembers which color must come next. Start from $(v, 0)$ and follow only the required color at each step. All vertices appearing in reachable states are exactly the vertices reachable by a French flag walk. The running time is $O(n + m)$.

Problem 7

Suppose we are given an undirected weighted graph G together with a minimum spanning tree T of G .

(a) Update the MST when the weight of a single edge e is decreased

Algorithm

Case 1: $e \in T$. If the decreased edge already belongs to the MST, then keep T unchanged.

Case 2: $e \notin T$.

1. Add $e = (u, v)$ to T . This creates exactly one cycle.
2. Find the maximum-weight edge f on that cycle.
3. If $w(e) < w(f)$, replace f by e :

$$T' = T - f + e.$$

4. Otherwise keep T unchanged.

Why this is correct.

Case 1: $e \in T$. Only the weight of a tree edge decreases. The same tree T is still spanning, and its total weight becomes smaller. Since no other spanning tree changed weight except possibly relative comparisons with T , T is still an MST.

Case 2: $e \notin T$. Adding e to T creates a unique cycle C . By the cycle property of MSTs, if e is lighter than the heaviest edge f on C , then no MST can contain f while excluding e from that cycle. Replacing f by e gives a spanning tree of smaller total weight, and it is an MST in the updated graph. If e is not lighter than the heaviest edge on the cycle, then replacing any edge on the cycle cannot improve the tree, so the old T remains an MST.

Running time. Finding the unique path between the endpoints of e in the tree T and locating the heaviest edge on that path takes $O(n)$ time using DFS/BFS on the tree. Therefore the update time is

$$O(n).$$

(b) Update the MST when the weight of a single edge e is increased

Algorithm

Case 1: $e \notin T$. If the increased edge is not in the MST, keep T unchanged.

Case 2: $e \in T$.

1. Remove e from T . This splits T into two connected components A and B .
2. Among all edges of G with one endpoint in A and the other in B , find a minimum-weight crossing edge f .
3. If $f = e$, keep T unchanged. Otherwise set

$$T' = T - e + f.$$

Why this is correct.

Case 1: $e \notin T$. Since e was not used in the MST, increasing its weight cannot make the current MST worse relative to trees that do not use e . So T remains an MST.

Case 2: $e \in T$. Removing e defines a cut (A, B) . In any MST of the updated graph, the edge chosen across this cut must be a minimum-weight edge crossing the cut, by the cut property. Therefore we scan all graph edges and choose the lightest edge f crossing between A and B . Replacing e by f gives exactly the correct updated MST.

Running time. Finding the two tree components after removing e takes $O(n)$ on the tree. Scanning all graph edges to find the lightest crossing edge takes $O(m)$. Thus the total update time is

$$O(m).$$

Final Answer

Decrease case:

- If $e \in T$, do nothing.
- If $e \notin T$, add it to T , find the heaviest edge on the created cycle, and replace that edge if it is heavier than e .

Increase case:

- If $e \notin T$, do nothing.
- If $e \in T$, remove it, which creates a cut, then insert the lightest edge crossing that cut.

The update times are $O(n)$ for a decrease and $O(m)$ for an increase.